LC 881 Boats to save people

people = [3, 2, 2, 1], limit = 3

→ Carry atmost 2 people, within weight limit

→ return minimum no. of boats

Sol:

→ Sort the array ⇒ [1, 2, 2, 3] (left → lightest, right → heaviest)

Two pointer

2 possibilities

1. lightest + heaviest ≤ limit
   put together move both
2. lightest + heaviest > limit
   add heaviest, move heaviest right —;

LC 253 Meeting Rooms II

Greedy + Sorting + Priority Queue

meetings = {(0,30) (5,10) (15,20)}

m_1 = 0 ----- 30 → Room 1

m_2 = 5-10 → Room 2 } both overlap

m_3 = 15-20 → Room 2 ⇒ Output: 2

After m_2 ends m_3 start

dont overlap

1. → Sord by start time
2. Meeting ending times stored in PriorityQueue
   Smallest at top ← [10, 30, 25]
   Earliest available room

Date :

Ex: (0,30) (5,10) (15,20) → Sorted ①

m_1 → Create room1 → PQ: 30

m_2 → Check earliest available room, Peek 30, meet start = 5 (30 <= 5 ✗)

Create another room

PQ: [10, 30] ← Top, 2 rooms

m_3 → (15,20) → Check earliest available room

Peek → 10, 10 <= 15 ✓ Start at 15

room is free, same room, return 2

Pop → 10, add → 20 to PQ: [20, 30]

LC 1353 → Maximum no. of events that can be attended

events = [1,2] [2,3] [3,4] ⇒ 3

Now: [(1,4) (1,2) (2,2) (3,3)]

which one to attend first?

(1,4) (1,2) → Same Day! Choose (1,2)

(1,4) can wait (1,2) can't wait long

Greedy:

"Among all events currently available attend event that ends earliest"

Date :

Why min-heap?

we attend the one ending early

min-heap will give

model:

Current day

↓

add all event that have started

↓

remove expired events

↓

which event ends earliest

↓

Attend it

↓

remove heap

↓

day++

events = [(1,2), (2,3), (3,4)]

	heap	Attend	Remove
Day 1: [1,2] =>	2	event ending at 2	heap: emp

count: 1

Day 2: Add events on Day 2 [2,3], heap: 3

count: 2

	Add	heap	count
Day 3: (3,4)	4	3	return: 3